Business Case Study SQL

1. Import the dataset and do usual exploratory analysis steps, like checking the structure & characteristics of the dataset:

Q1). Data type of all columns in the "customers" table.
Answer :-

customers

QUERY SHARE COPY SNAPSHOT DELETE EXPORT REFRESH

SCHEMA DETAILS PREVIEW TABLE EXPLORER PREVIEW INSIGHTS PREVIEW LINEAGE DATA PROFILE DATA QUALITY

Filter Enter property name or value

Field name	Type	Mode	Key	Collation	Default value	Policy tags	Description
customer_id	STRING	NULLABLE	-	-	-	-	-
customer_unique_id	STRING	NULLABLE	-	-	-	-	-
customer_zip_code_prefix	INTEGER	NULLABLE	-	-	-	-	-
customer_city	STRING	NULLABLE	-	-	-	-	-
customer_state	STRING	NULLABLE	-	-	-	-	-

Q2). Get the time range between which the orders were placed.
Answer :-

```
SELECT MIN(order_purchase_timestamp) AS start_date, MAX(order_purchase_timestamp)
AS end_date
FROM BCS.orders;
```

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	start_date	end_date
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Explanation - To find the time range during which orders were placed, we use the `MIN(order_purchase_timestamp)` function to get the earliest order date and `MAX(order_purchase_timestamp)` to get the latest order date. This gives us the start and end dates for when orders were placed, showing us the full period covered by the data.

Q3). Count the Cities & States of customers who ordered during the given period.
Answer :-

```
SELECT COUNT(DISTINCT customer_city) AS no_of_cities,
COUNT(DISTINCT customer_state) AS no_of_states
FROM BCS.customers;
```

Explanation - To find out how many unique cities and states have customers who placed orders, we use the `COUNT(DISTINCT customer_city)` function to count all the different cities and `COUNT(DISTINCT customer_state)` to count all the different states. This tells us the total number of unique cities and states where our customers are located.

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	no_of_cities	no_of_states
1	4119	27

2. In-depth Exploration:

Q1. Is there a growing trend in the no. of orders placed over the past years?
Answer:-

```
SELECT EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
COUNT(*) AS total_orders
FROM BCS.orders
GROUP BY year
ORDER BY year;
```

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	year	total_orders
1	2016	329
2	2017	45101
3	2018	54011

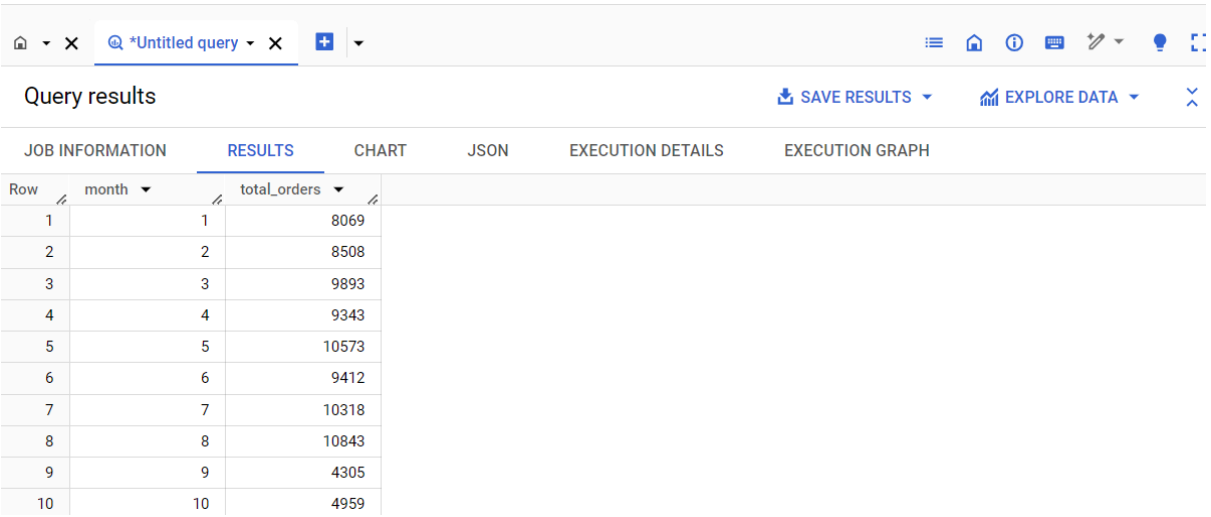
Explanation -To check if the number of orders is growing each year, we start by extracting the year from the order timestamps using `EXTRACT(YEAR FROM order_purchase_timestamp)`. Then, we count the total orders for each year with `COUNT(*)`. Finally, we group the results by year and sort them to see if the number of orders has been increasing over the years.

Q2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Answer :-

```
SELECT EXTRACT(MONTH FROM order_purchase_timestamp) AS month, COUNT(*) AS
            total_orders
FROM BCS.orders
GROUP BY month
ORDER BY month;
```

Explanation - To find out how many orders are placed each month, we first extract the month from the order timestamps using the `EXTRACT(MONTH FROM order_purchase_timestamp)` function. We then count the total number of orders for each month with `COUNT(*)`. Finally, we group the results by month and sort them to see the order volume month by month.



Row	month	total_orders
1	1	8069
2	2	8508
3	3	9893
4	4	9343
5	5	10573
6	6	9412
7	7	10318
8	8	10843
9	9	4305
10	10	4959

Q3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
0-6 hrs : Dawn
7-12 hrs : Mornings
13-18 hrs : Afternoon
19-23 hrs : Night

Answer -

```
SELECT
CASE
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN 'Night'
END AS time_of_day,
COUNT(*) AS order_count
FROM
BCS.orders
GROUP BY
time_of_day;
```

Press Alt+F1 for accessibility opti

Query results

SAVE RESULTS EXPLORE DATA

JOB INFORMATION RESULTS CHART JSON EXECUTION DETAILS EXECUTION GRAPH

Row	time_of_day	order_count
1	Morning	27733
2	Dawn	5242
3	Afternoon	38135
4	Night	28331

Explanation - To find out when Brazilian customers place orders most often, we extract the hour from the order timestamps and categorize it into Dawn, Morning, Afternoon, or Night using a `CASE` statement. We then count the number of orders in each time block and group by these categories to see the peak ordering times.

3. Evolution of E-commerce orders in the Brazil region:

Q1. Get the month on month no. of orders placed in each state.
Answer -

```
SELECT
CASE
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN
'Afternoon'
 WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23 THEN 'Night'
END AS time_of_day,
COUNT(*) AS order_count
FROM
 BCS.orders
GROUP BY
 Time_of_day;
```

Query results

JOB INFORMATION | RESULTS | CHART | JSON | EXEC

Row	time_of_day	order_count
1	Morning	27733
2	Dawn	5242
3	Afternoon	38135
4	Night	28331

Explanation :- To track the number of orders placed each month in each state, we first extract the Order count for each Time of the day from the order dates using the `EXTRACT` function. Then, we count the number of orders for each Time of day using the `COUNT` function. Grouping by Mornig, Dawn, Afternoon, Night helps us see how orders vary.

Q2. How are the customers distributed across all the states?
Answer -

```
SELECT
customer_state,
COUNT(*) AS customer_count
FROM
BCS.customers
GROUP BY
customer_state
ORDER BY
customer_count DESC;
```

*Untitled query orders

Query results

JOB INFORMATION RESULTS CHART JSOI

Row	customer_state	customer_count
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020

Explanation :- To see how customers are spread across different states, we count the number of customers in each state using the `COUNT` function. We group the results by state to get the total number of customers for each location. This gives us a clear picture of how customer numbers are distributed across all the states.

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

Q1. Get the % increase in the cost of orders from 2017 to 2018 (include months between Jan to Aug only).

Answer -

```
SELECT
(SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018 THEN
p.payment_value ELSE 0 END) -
SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 THEN
p.payment_value ELSE 0 END)) /
NULLIF(SUM(CASE WHEN EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2017 THEN
p.payment_value ELSE 0 END), 0) * 100 AS percentage_increase
FROM
  BCS.orders o
JOIN
  BCS.payments p ON o.order_id = p.order_id
WHERE
  EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
  AND EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017, 2018);
```

Query results

JOB INFORMATION | RESULTS | CH

Row	percentage_increase
1	136.9768716466...

Explaination :- To find the percentage increase in order costs from 2017 to 2018, we sum up the costs from January to August for each year. Then, we calculate the percentage change between these totals to see how much costs have risen.

Q2. Calculate the Total & Average value of order price for each state?

```
SELECT
c.customer_state AS state,
SUM(oi.price) AS total_price,
AVG(oi.price) AS avg_price
FROM
BCS.order_items oi
JOIN
BCS.orders o ON oi.order_id = o.order_id
JOIN
BCS.customers c ON o.customer_id = c.customer_id
GROUP BY
c.customer_state
ORDER BY
total_price DESC;
```

Row	state	total_price	avg_price
1	SP	5202955.050001...	109.6536291597...
2	RJ	1824092.669999...	125.1178180945...
3	MG	1585308.029999...	120.7485741488...
4	RS	750304.0200000...	120.3374530874...
5	PR	683083.7600000...	119.0041393728...
6	SC	520553.3400000...	124.6535775862...
7	BA	511349.9900000...	134.6012082126...
8	DF	302603.9399999...	125.7705486284...
9	GO	294591.9499999...	126.2717316759...
10	ES	275037.3099999...	121.9137012411...

Explanation :- To find the total and average order price for each state, we first use the `SUM` function to add up all the order prices in each state. Then, we calculate the average order price per state using the `AVG` function. This gives us a visibility of both the total spending and the average price of orders in each state.

Q3. Calculate the Total & Average value of order freight for each state.

```
SELECT
c.customer_state AS state,
SUM(oi.freight_value) AS total_freight,
AVG(oi.freight_value) AS avg_freight
FROM
BCS.order_items oi
JOIN
BCS.orders o ON oi.order_id = o.order_id
JOIN
BCS.customers c ON o.customer_id = c.customer_id
GROUP BY
c.customer_state
ORDER BY
total_freight DESC;
```

*Untitled query order_items

Query results

JOB INFORMATION RESULTS CHART JSON EXECUTIO

Row	state	total_freight	avg_freight
1	SP	718723.0699999...	15.14727539041...
2	RJ	305589.3100000...	20.96092393168...
3	MG	270853.4600000...	20.63016680630...
4	RS	135522.7400000...	21.73580433039...
5	PR	117851.6800000...	20.53165156794...
6	BA	100156.6799999...	26.36395893656...
7	SC	89660.26000000...	21.47036877394...
8	PE	59449.65999999...	32.91786267995...
9	GO	53114.97999999...	22.76681525932...
10	DF	50625.49999999...	21.04135494596...

Explanation :- To find the total and average freight costs per state, we use the `SUM` function to add up all freight costs and the `AVG` function to get the average. This shows us both the total and average shipping expenses for each state.

5. Analysis based on sales, freight and delivery time.

Q1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.
Also, calculate the difference (in days) between the estimated & actual delivery date of an order.
Do this in a single query.

Answer :-

```
SELECT
o.order_id,
DATE_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY) AS
days_to_deliver,
DATE_DIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date, DAY) AS
delivery_diff
FROM
BCS.orders o;
```

Query results

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETA

Row	order_id	days_to_deliver	delivery_diff
1	1950d777989f6a877539f5379...	30	12
2	2c45c33d2f9cb8ff8b1c86cc28...	30	-28
3	65d1e226dfaeb8cdc42f66542...	35	-16
4	635c894d068ac37e6e03dc54e...	30	-1
5	3b97562c3aee8bdedcb5c2e45...	32	0
6	68f47f50f04c4cb6774570cfde...	29	-1
7	276e9ec344d3bf029ff83a161c...	43	4
8	54e1a3c2b97fb0809da548a59...	40	4
9	fd04fa4105ee8045f6a0139ca5...	37	1
10	302bb8109d097a9fc6e9cefc5...	33	5
11	66057d37308e787052a32828...	38	6

Explanation: - To find out how long each order took to deliver from the purchase date, we calculate the difference between the actual delivery date and the purchase date using the `DATE_DIFF` function. We also find out how far off the actual delivery was from the estimated delivery date by comparing these two dates with `DATE_DIFF` again. This gives us a clear view of both the total delivery time and the accuracy of our delivery estimates.

Q2. Find out the top 5 states with the highest & lowest average freight value.

Answer

```
(
SELECT
c.customer_state AS state,
AVG(oi.freight_value) AS avg_freight
 FROM
 BCS.order_items oi
 JOIN
 BCS.orders o ON oi.order_id = o.order_id
 JOIN
 BCS.customers c ON o.customer_id = c.customer_id
 GROUP BY
 c.customer_state
 ORDER BY
 avg_freight DESC
 LIMIT 5
)
UNION ALL
(
SELECT
 c.customer_state AS state,
 AVG(oi.freight_value) AS avg_freight
 FROM
 BCS.order_items oi
 JOIN
 BCS.orders o ON oi.order_id = o.order_id
 JOIN
 BCS.customers c ON o.customer_id = c.customer_id
 GROUP BY
 c.customer_state
 ORDER BY
 avg_freight ASC
 LIMIT 5
);
```

Explanation - To find the top 5 states with the highest and lowest average freight values, we calculate the average shipping cost for each state using `AVG`. We then sort the states by these averages and pick the top 5 with the highest and lowest costs. This helps us see where shipping is most and least expensive.

JOB INFORMATION | RESULTS | CHART | JSON

Row	state	avg_freight
1	RR	42.98442307692...
2	PB	42.72380398671...
3	RO	41.06971223021...
4	AC	40.07336956521...
5	PI	39.14797047970...
6	SP	15.14727539041...
7	PR	20.53165156794...
8	MG	20.63016680630...
9	RJ	20.96092393168...
10	DF	21.04135494596...

Q3. Find out the top 5 states with the highest & lowest average delivery time.

Answer -

```
(
SELECT
 c.customer_state AS state,
 AVG(DATE_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY))
AS avg_delivery_time,
 'Highest' AS category
FROM
 BCS.orders o
JOIN
 BCS.customers c ON o.customer_id = c.customer_id
GROUP BY
 c.customer_state
ORDER BY
```

```sql
 avg_delivery_time DESC
LIMIT 5
)
UNION ALL
(
SELECT
 c.customer_state AS state,
 AVG(DATE_DIFF(o.order_delivered_customer_date, o.order_purchase_timestamp, DAY))
AS avg_delivery_time,
 'Lowest' AS category
FROM
 BCS.orders o
JOIN
 BCS.customers c ON o.customer_id = c.customer_id
GROUP BY
 c.customer_state
ORDER BY
 avg_delivery_time ASC
LIMIT 5
);
```

*Untitled query order_items

Query results

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS

Row	state	avg_delivery_time	category
1	RR	28.97560975609...	Highest
2	AP	26.73134328358...	Highest
3	AM	25.98620689655...	Highest
4	AL	24.04030226700...	Highest
5	PA	23.31606765327...	Highest
6	SP	8.298061489072...	Lowest
7	PR	11.52671135486...	Lowest
8	MG	11.54381329810...	Lowest
9	DF	12.50913461538...	Lowest
10	SC	14.47956019171...	Lowest

Explanation - To find the top 5 states with the highest and lowest average delivery times, we calculate the delivery time for each order using `DATE_DIFF` and then average these times by state. We sort the states by these average times and select the top 5 with the longest and shortest delivery periods. This shows us where deliveries are the slowest and fastest.

Q4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Answer -

```
SELECT
 c.customer_state AS state,
 AVG(DATE_DIFF(o.order_estimated_delivery_date, o.order_delivered_customer_date,
DAY)) AS avg_delivery_diff
FROM
 BCS.orders o
JOIN
 BCS.customers c ON o.customer_id = c.customer_id
GROUP BY
 c.customer_state
ORDER BY
 avg_delivery_diff DESC
LIMIT 5;
```

*Untitled query order_items

Query results

JOB INFORMATION RESULTS CHART JSON EX

Row	state	avg_delivery_diff
1	AC	19.76250000000...
2	RO	19.13168724279...
3	AP	18.73134328358...
4	AM	18.60689655172...
5	RR	16.41463414634...

Explanation - To find the top 5 states where deliveries are much faster than estimated, we start by joining the orders with the customers' data to get the state for each order. Next, we calculate how many days earlier each delivery was compared to what was estimated using the `DATE_DIFF` function. We then average these differences for each state to see which states consistently have quicker deliveries. Finally, we sort the results by the largest average difference (showing the fastest deliveries) and pick the top 5 states. This way, we can spot where deliveries are exceeding expectations the most.

6. Analysis based on the payments:

Q1. Find the month on month no. of orders placed using different payment types.
Answer -

```
SELECT
 EXTRACT(YEAR FROM o.order_purchase_timestamp) AS year,
 EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
 p.payment_type,
 COUNT(DISTINCT o.order_id) AS order_count
FROM
 BCS.orders o
JOIN
 BCS.payments p ON o.order_id = p.order_id
```

```
GROUP BY
 year, month, p.payment_type
ORDER BY
 year, month, p.payment_type;
```

*Untitled query | order_items

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | CHART | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	year	month	payment_type	order_count
1	2016	9	credit_card	3
2	2016	10	UPI	63
3	2016	10	credit_card	253
4	2016	10	debit_card	2
5	2016	10	voucher	11
6	2016	12	credit_card	1
7	2017	1	UPI	197
8	2017	1	credit_card	582
9	2017	1	debit_card	9
10	2017	1	voucher	33

Load more

Explanation :- To find out the number of orders placed each month using different payment types, we start by extracting the year and month from the order dates so we can organize the data by time. Next, we join this order information with the payment details to connect each order with its payment method. Finally, we count the number of orders for each payment type in each month. This way, we can see how the popularity of different payment methods changes over time.

Q2. Find the no. of orders placed on the basis of the payment installments that have been paid.

Answer -

```
SELECT
 p.payment_installments,
 COUNT(DISTINCT p.order_id) AS order_count
FROM
 BCS.payments p
GROUP BY
 p.payment_installments
ORDER BY
 p.payment_installments;
```

*Untitled query payments

Query results

JOB INFORMATION RESULTS CHART

Row	payment_installment	order_count
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

Explanation :- We want to figure out how many orders were placed based on how many installments were paid. So, we count the number of unique orders for each type of paymenti installment plan, like 1-month, 3-month, or 6-month payments. This helps us see how often each installment option is used.